# Theory of Computation

BSc Computer Science - Durham University

Matthew Emerson

October 2026

# Contents

**1 Module Introduction** **1**
1.1 Assessment . . . 1
1.2 Timetable . . . 1

**2 Models of Computation** **1**
2.1 Finite-State Automata and Regular Languages . . . 1
2.2 Büchi Automata and Omega-Regular Languages . . . 5
2.3 Linear-Time Logic and Translating Into Büchi Automata . . . 6

**3 Algorithms and Complexity** **6**
3.1 Asymptotic Complexity and Notation . . . 6
3.2 Greedy Algorithms . . . 6
3.3 Divide and Conquer Algorithms . . . 7
3.4 Dynamic Programming Algorithms . . . 7
3.5 Machinings in Graphs . . . 7

**I Appendix** **8**

**4 Bibliography** **8**
4.1 Module Professors . . . 8
4.2 References . . . 8
4.3 List of Figures . . . 8
4.4 Recommended Reading . . . 8

# 1 Module Introduction

Theory of Computation has the module code COMP2181 and is led by Dr Barnaby Martin.

## 1.1 Assessment

This module is assessed through two summative benchtests, and one final exams, these can be seen in Table 1. Additional formatives are also shown in italics.

| Title | Opening | Due | Feedback | Professor | Weight |
|---|---|---|---|---|---|
| Benchtest 1 | Week 11 | - | 2nd Feb | Dr Dantchev and Dr Mertzios | 12.5% |
| Benchtest 2 | Week 20 | - | - | Dr Dantchev and Dr Mertzios | 12.5% |
| Final Exam | May Exam | - | - | Dr Barnaby Martin | 75% |

Table 1: Assessments for Theory of Computation.

## 1.2 Timetable

This module is taught through two one-hour lectures and one two-hour computer class each week as seen below in Table 2.

| Day | Time | Class Type | Location | Lecturer |
|---|---|---|---|---|
| Tuesday | 10:00 - 11:00 | Lecture | CLC202 | Dr Barnaby Martin |
| Wednesday | 09:00 - 11:00 | Problem Class | TLC116 | - |
| Thursday | 14:00 - 15:00 | Lecture | CLC202 | Dr Sukanya Pandey |

Table 2: Michaelmas Term Timetable for Theory of Computation.

# 2 Models of Computation

This section is taught by Dr Barnaby Martin. Check for recommended reading.

## 2.1 Finite-State Automata and Regular Languages

### 2.1.1 Formal Definitions

**Definition 2.1.1.1**: A deterministic finite-state automaton is a five-tuple $(Q, \Sigma, \delta, q_0, F)$ where $Q$ is the finite set of states, $\Sigma$ is a finite alphabet, $\delta : Q \times \Sigma \to Q$ is the transition function, $q_0 \in Q$ is the start state, and $F \subseteq Q$ is the set of accept states.

**Definition 2.1.1.2**: A DFA $M$ recognises the language $L$ if $L = \{w | M \text{ accepts } W\}$.

**Definition 2.1.1.3**: A language is called a regular language if some DFA recognises it.

**Definition 2.1.1.4**: A regular expression $R$ defines a language $L(R)$. We shall eventually prove that $RE \equiv DFA$. $R$ is a regular expression over the alphabet $\Sigma$ if $R$ is:

1. $a$ for some $a \in \Sigma$.
2. $\epsilon$ (the empty string).
3. $\emptyset$ (the empty set).
4. $(R_1 \cup R_2)$ where $R_1$ and $R_2$ are regular expressions.
5. $R_1 \cdot R_2$ where $R_1$ and $R_2$ are regular expressions.
6. $R_1^*$ where $R_1$ is a regular expression.


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

The list of regular operations can be seen in Table 3 where the operations labelled with $^+$ are language theory specific.

| Operation Name | Notation | Set Equivalent |
|---|---|---|
| Union | $A \cup B$ | $\{x \mid x \in A \text{ or } x \in B\}$ |
| Intersection | $A \cap B$ | $\{x \mid x \in A \text{ and } x \in B\}$ |
| Difference | $A \backslash B$ | $\{x \mid x \in A \text{ and } x \notin B\}$ |
| Complement | $\overline{A}$ | $\Sigma * \backslash A$ |
| Concatenation$^+$ | $A \circ B$ | $\{xy \mid x \in A \text{ and } y \in B\}$ |
| Star$^+$ | $A*$ | $\{x_1 x_2 ... x_k \mid k \geq 0 \text{ and } x_i \in A \text{ for every } i, 1 \leq i \leq k\} \equiv \cup_{i=0}^{\infty} A^i$ |

Table 3: Regular Operations.

### 2.1.2 Combining Automata

Sometimes we may want to implement an operation on multiple languages. We can do this by combining their automata. If we have that $L_1$ is recognised by $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and $L_2$ is recognised by $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$; we want to combine $M_1$ and $M_2$ into a new automaton $M$ that would recognise $L_1 \cup L_2$.

You would think that we could just run the automata sequentially, and accept if either $M_1$ or $M_2$ accept, but this does not work. This is because after $M_1$ has run on the input, the input is exhausted and there is no way to rewind it in order to run $M_2$.

Instead, we can attempt to run $M_1$ and $M_2$ on the input in parallel. We do this by using pairs of states (one for each DFA). Formally, the components of the new automaton $M = (Q, \Sigma, \delta, q_0, F)$ are:

1. The set of possible states contains every possible cross-set pair of $Q_1$ and $Q_2$.

$$Q = Q_1 \times Q_2$$

2. For every $s \in Q_1, u \in Q_2, a \in \Sigma$. The transition function contains every transition from a pair of states from $Q$ being given any input from $\Sigma$.

$$\delta((s, u), a) = (\delta_1(s, a), \delta(u, a))$$

3. The start state is the pair containing both start states from $M_1$ and $M_2$.

$$q_0 = (q_1, q_2)$$

4. The set of accept states is every possible cross-set pair of accept states from $F_1$ and $F_2$.

$$F = \{(s, u) \in Q \mid s \in F_1 \text{ or } u \in F_2\}$$

### 2.1.3 Non-Deterministic Finite Automaton

If, instead you wish to work with concatenation, such that $w \in L_1 \circ L_2$ only if there are words $x$ and $y$ such that $w = xy$, $x \in L_1$ and $y \in L_2$. You need to run $M_1$ on a prefix of $w$ and then run $M_2$ on the rest. This must be done non-deterministically and stops becoming a DFA as all accept states of $M_1$ have to map to the start state of $M_2$.

**Definition 2.1.3.1**: A non-deterministic finite automaton is a five-tuple $(Q, \Sigma, \delta, q_0, F)$, where $Q$ is the finite set of states, $\Sigma$ is a finite alphabet, $\delta : Q \times (\Sigma \cup \{\epsilon\}) \to \mathscr{P}(Q)$ is the transition function, $q_0 \in Q$ is the start state, and $F \subseteq Q$ is the set of accept states.


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

**Computing NFA**

Before computing an NFA, an important convention to note is that a single step in a computation consists of first reading the next input symbol followed by taking any number of $\epsilon$-transitions. This means that empty strings $\epsilon$s can be freely added inside the actual word $\in \Sigma*$, e.g., 010110 can be viewed as, say, $\epsilon 01\epsilon\epsilon 01\epsilon 01$.

Let $M = (Q, \Sigma, \delta, q_0, F)$ be an NFA and $w = w_1 w_2 ... w_n$ be a word over $\Sigma \cup \{\epsilon\}$. $M$ accepts $w$ if there exists a sequence of states $r_0, r_1, r_2, ..., r_n$ satisfying the following conditions:

1. $r_0 = q_0$.
2. $r_{i+1} \in \delta(r_i, w_{i+1})$ for every $i, 0 \leq i \leq n-1$.
3. $r_n \in F$.

### 2.1.4 Converting NFA into DFA

The difference between a DFA and NFA is that a DFA has a unique computation (path) and either accepts or rejects, while an NFA generally has many computation paths and accepts if at least one computation accepts and only rejects if all the computations reject.

**Theorem 2.1.4.1**: Every NFA has an equivalent DFA.

*Proof*: To formally convert an NFA to a DFA, let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA recognising some language $L$ (assume that $N$ has no $\epsilon$-transitions). We shall construct a DFA $M = (Q', \Sigma, \delta', q_0', F')$ that recognises the same language $L$ as follows.

1. $Q' = \mathscr{P}(Q)$.
2. For $R \in Q'$ and $a \in \Sigma$, let

$$\delta'(R, a) = \{q \in Q | q \in \delta(r, a) \text{ for some } r \in R\} \tag{1}$$

   equivalently,

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a) \tag{2}$$

3. $q_0' = \{q_0\}$.
4. $F' = \{R \in Q' | \exists r \in R, r \in F\}$.

Now, to consider $\epsilon$-transitions, for $R \subseteq Q$, let

$$E(R) = \{q | q \text{ is reachable from } R \text{ through a number of } \epsilon\text{-transitions.}\} \tag{3}$$

Now, taking the above steps, we modify (1) to

$$\delta'(R, a) = \{q \in Q | q \in E(\delta(r, a)) \text{ for some } r \in R\} \tag{4}$$

and (2) to

$$\delta'(R, a) = \bigcup_{r \in R} E(\delta(r, a)) \tag{5}$$

We then modify step 3 to say $q_0' = E(\{q_0\})$.

□

**Corollary 2.1.4.1**: A language is regular iff some NFA recognises it.

*Proof*: See Theorem 2.1.4.1.

□


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

**Theorem 2.1.4.2**: A language is regular iff some regular expression describes it.

*Proof*: $\Leftarrow$ Given a regular expression $R$, construct a NFA that recognises $L(R)$ by structural induction:

- *Base Case*: All initial regular expressions can be implemented by NFAs.
- *Inductive Step*: We have already proven that the class of regular languages is closed under regular operations.

$\Rightarrow$ Given a DFA $M$, construct a regular expression that recognises $L(M)$.

1. Convert the given $n$-state DFA into an equivalent $n+2$ state GNFA.
2. Eliminate all the internal states of the GNFA one-by-one, while preserving the language recognised by the automaton.
   (a) Pick an internal state $r$ for elimination.
   (b) For every two states $u$ and $v$ with transitions between them and $r$, $u \neq r, v \neq r$, combine the regular expressions of the three into one, new expression.
3. $L(R)$ should now equal $L(M)$.

□

**Definition 2.1.4.1**: A generalised non-deterministic finite automaton is defined as having a single start state with no incoming edges, a single end state with no outgoing edges, and every transition being labelled with a regular expression.

### 2.1.5 Non-Regular Expression

**Definition 2.1.5.1**: A non-regular expression is a language which cannot be recognised by any DFA. We prove a language is non-regular by assuming there is a DFA that recognises language $L$ and then use an adversary argument against that DFA to get a contradiction.

**Lemma 2.1.5.1**: The pumping lemma states that, for every regular language $L$, there is a number $p$ (the pumping length of $L$) such that every word $w \in L$ of length at least $p$ can be divided into three parts $w = xyz$, satisfying the following conditions:

1. $xy^iz \in L$ for every $i \in N$ (in particular $xz \in L$ as one can take $i = 0$).
2. $y$ is a non-empty string.
3. The length of the string $xy$ is not greater than $p$.

*Proof*: Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA that recognises $L$, and let us set the pumping length $p$ to be the number of states in $M : p = |Q|$. Consider the computation of $M$ on the word $w = w_1w_2...w_n$ where $n \geq p$.

$$q_0 = r_0 \rightarrow w_1 \rightarrow r_1 \rightarrow w_2 \rightarrow ...r_{n-1} \rightarrow w_n \rightarrow r_n \in F \tag{6}$$

Let $i$ be the first moment in time when we see a repetition in the sequence of states $r_0, r_1, ..., r_n$ (such an $i$ must exist by the Pigeon-Hole principle, as $n+1 > p$). i.e., $r_i = r_j$ for some $i > j$ and the states $r_0, r_1, ..., r_{i-1}$ are all different. We can then set

$$x = w_1w_2...w_j \quad y = w_{j+1}...w_i \quad z = w_{i+1}...w_n \tag{7}$$

It is now straightforward to check that all three conditions are met.

□


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

**Example 2.1.5.1**: Prove that the language $L = \{0^n 1^n | n \in \mathbb{N}\}$ is not regular.

*Proof*: Take the word $0^p 1^p \in L$ where $p$ is the pumping length of $L$. If $0^p 1^p = xyz$ and $|xy| \leq p$, then $x = 0^i, y = 1^j$ and $z = 0^{p-i-j}1^p$. But then $xy^2z = 0^{p+j}1 \notin L$ as $|y| > 0$, i.e., $j > 0$.

□

**Definition 2.1.5.2**: An algorithm that minimises any given DFA, i.e., finds an equivalent DFA with the minimal number of states. It answers both "What is the smallest DFA that recognises the same language?" and "Given two DFAs, do they recognise the same language?". It can be achieved by removing any states not reachable from the start state and contracting a set of states that are equivalent into a single state.

**Definition 2.1.5.3**: Two states $s$ and $t$ are equivalent if any word $w$, which is accepted when starting from $s$, is also accepted when starting from $t$.

**Definition 2.1.5.4**: A extended transition function is defined by letting a DFA $M = (Q, \Sigma, \delta, q_0, F)$ be given, and let $\hat{\delta} : Q \times \Sigma^* \to Q$ be defined as:

$$\hat{\delta}(s, \epsilon) = s \tag{8}$$

$$\hat{\delta}(s, w_1 w_2 ... w_n) = \hat{\delta}(\delta(s, w_1), w_2, ..., w_n) \tag{9}$$

for every state $s$ and every word $w_1 w_2 ... w_n$.

## 2.2 Büchi Automata and Omega-Regular Languages

### 2.2.1 Finite-State Automata Over Infinite Words

**Definition 2.2.1.1**: If we have a NFA where the input is infinite (i.e., $\Sigma^\omega$), we have a Büchi automaton. A Büchi automaton accepts $w_1 w_2 ... \in \Sigma^\omega$ if there is a sequence of states $r_0, r_1, r_2, ... \in Q^\omega$ satisfying the following conditions:

- $r_0 = q_0$.
- $(r_i, w_{i+1}, r_{i+1}) \in \Delta$ for every $i > 0$.
- There are infinitely many $r_i$'s in $F$.

**Definition 2.2.1.2**: An $\omega$-regular language/expression is build upon regular languages, using the following operations:

- $A \cup B$, where both $A$ and $B$ are $\omega$-regular.
- $AB$, where $A$ is regular, and $B$ is $\omega$-regular.
- $A^\omega$, which is $\{a_1 ... | a_i \in A\}$, i.e., an infinite sequence of words from $A$, where $A$ is regular and doesn't contain the empty word.

Let $A$ be a regular language. The limit of $A$, $\lim A$ is the language $\{a \in \Sigma^\omega | a$ has infinitely many prefixes in $A\}$.

$L_1 = \{a \in \Sigma^\omega | a$ contains infinitely many 0s.$\}$.
$L_2 = \{a \in \Sigma^\omega | a$ contains finitely many 1s.$\}$.

**Proposition 2.2.1.1**: $L_2$ cannot be recognised by a deterministic Büchi automaton.

*Proof*: Assume that $L_2$ can be recognised by some deterministic Büchi automaton $M$. We will construct a word accepted by $M$ that contains infinitely many 1s and thus doesn't belong to $L_2$.

1. $0^\omega \in L_2$ must be accepted by $M$, so there is some finite prefix $0^{n_1}$ that leads into accepting state $p_1$.
2. $0^{n_1}10^\omega \in L_2$ must be accepted by $M$, so there is some finite prefix $0^{n_1}10^{n_2}$ that leads into accepting state $p_2$.

3. $0^{n_1}10^{n_2}10^{\omega} \in L_2$ must be accepted by $M$, so there is some finite prefix $0^{n_1}10^{n_2}10^{n_3}$ that leads into accepting state $p_3$.
4. ...

The limit of the infinite process is a word of the form $(0^*1)^{\omega} \notin L_2$ that is accepted by $M$ since it visits an infinite sequence of accepting states $p_1, p_2, ....$

□

## 2.3 Linear-Time Logic and Translating Into Büchi Automata

# 3 Algorithms and Complexity

This section is taught by Dr Sukanya Pandey. The recommended reading for this topic is:

- Introduction to Algorithms (Cormen T.H., et al., 2009).

## 3.1 Asymptotic Complexity and Notation

Lecture one was just a recall of some of last year's Algorithms and Data Structures module.

## 3.2 Greedy Algorithms

**Definition 3.2.0.1**: The knapsack problem is defined as one of two variants. Either:

- The 0-1 knapsack problem (the classic variant), where a thief robs a store and finds $n$ items. Item $i$ is worth $v_i$ euros and weighs $w_i$ kilos. The thief can carry at most $W$ kilos in his knapsack. *Which items can he take to maximise his profit?*
- the fractional knap problem, which has the same setup, but this time the thief can take fractions of items.
- We can assume (without loss of generality) that the knapsack is not big enough to carry all items:

$$W < w_1 + w_2 + ... + w_n \tag{10}$$

### 3.2.1 The Greedy Approach

To solve the fractional problem, the greedy approach would be to calculate the value per kilo using (11) for each item. You then sort the items by value per kilo, in decreasing order. You then take as much of the first item as possible, then the second item, etc., until you reach the capacity of the knapsack $W$.

$$r_i = \frac{v_i}{w_i} \tag{11}$$

**Example 3.2.1.1**: For example, if $W = 10$, and $\mathbf{v} = \{4, 10, 12, 20\}$ and $\mathbf{w} = \{4, 2, 8, 5\}$. We can then calculate $\mathbf{r} = \{1, 5, 1.5, 4\}$. The greedy algorithm then dictates that you should take all of item 2, all of item 4, and 3 kilos of item 3 $W = 2 + 5 + 3$. This gives a total value of $V = 10 + 20 + (3 \times 1.5) = 34.5$.

The running time of this algorithm can be calculated by taking that:

1. Step 1, calculating the value per kilo takes $\Theta(n)$.
2. Step 2, sorting the value per kilo array takes $\Theta(n \; log \; n)$.
3. Step 3, selecting the items can be done in $\Theta(n)$.

Therefore, the worst-case running time is $\Theta(n \; log \; n)$.

However, approach will not work for the 0-1 knapsack problem.


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

### 3.2.2 Correctness of the Greedy Algorithm for Fractional Knapsack

$n$ items are sorted by value per kilo, (11) in decreasing order, i.e.,

$$r_1 \geq r_2 \geq ... \geq r_n \tag{12}$$

Two items, $p$ and $q$ with $r_p = r_q$ are considered to be equivalent. Hence, we may assume that

$$r_1 > r_2 > ... > r_n \tag{13}$$

The total available amount of item $i$ is $w_i$. Let $k$ be such that

$$w_1 + w_2 + ... + w_k \leq W \tag{14}$$

and

$$w_1 + w_2 + ... + w_{k+1} > W \tag{15}$$

Let $\mathbf{x} = (x_1, x_2, ..., x_n)$ denote a solution: $0 \leq x_i \leq w_i$ is the amount the thief takes of item $i$ for $i = [0, n]$. The total value taken is then given by

$$\sum_{i=1}^{n} r_i x_i \tag{16}$$

The greedy solution $\mathbf{x}$ is given by $x_i = w_i$ for $1 \leq i \leq k$ and

$$x_{k+1} = W - (w_1 + w_2 + ... + w_k) \tag{17}$$

$x_i = 0$ for $k + 2 \leq i \leq n$. Let $\mathbf{x}^* = (x_1^*, x_2^*, ..., x_n^*)$ be an optimal solution.

Suppose that $\mathbf{x}^* \neq \mathbf{x}$. Then

- $x_i^* < x_i$ for some $i \leq k + 1$.
- Let $j = \min\{i : x_i^* < x_i\}$.
- And $x_l^* > x_l$ for some $l > j$.

Let us add some more of item $j$ to the optimal choice. Let $y$ be another solution, defined by

$$y = \begin{cases} y_j = \min\{w_j, x_j^* + (x_l^* - x_l)\} \\ y_l = x_l^* - (y_j - x_j^*) \\ y_i = x_i^* \text{ otherwise.} \end{cases} \tag{18}$$

## 3.3 Divide and Conquer Algorithms

## 3.4 Dynamic Programming Algorithms

## 3.5 Machinings in Graphs


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

# Part I
# Appendix

## 4 Bibliography

### 4.1 Module Professors

**Module Lead**: Doctor Barnaby Martin

- Focus: Complexity theory, finite model theory, and the links between logic and complexity.
- Email: barnaby.d.martin@durham.ac.uk
- Office Hours: Flexible, contact to arrange.

**Lecturer**: Doctor Sukanya Pandey

- Focus: Graph theory.
- Email: sukanya.pandey@durham.ac.uk
- Office Hours: Flexible, contact to arrange.

**Lecturer**: Doctor Stefan Dantchev

- Focus: Combinatorics.
- Email: s.s.dantchev@durham.ac.uk
- Office Hours: To be confirmed.

**Lecturer**: Doctor George Mertzios

- Focus: Algorithms and networks.
- Email: george.mertzios@durham.ac.uk
- Office Hours: To be confirmed.

### 4.2 References

### 4.3 List of Figures

### 4.4 Recommended Reading

#### 4.4.1 Algorithms and Complexity

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2009). Introduction to Algorithms. 3rd edn. MIT Press.